RATE LIMITER

Complete Interview Revision Guide

From fundamentals to distributed-system design, Redis, algorithms, race conditions, scaling, failure handling, and interview-ready answers.

How to use: Read once for understanding. Before an interview, revise the cheat sheets, algorithm comparison, system-design flow, and Q&A; section.

1. One-Page Mental Model

Rate limiting = controlling how much traffic a client or class of clients can send to a system during a period of time.

```
Client
  |
  v
CDN / Load Balancer
  |
  v
API Gateway / Rate Limiter
  |
  +----> Shared state (often Redis)
  |
  v
Application Servers
  |
  v
Database / Downstream services
```

Core idea: before allowing a request, the limiter checks whether the requester still has permission/quota to consume another request.

When limit is exceeded: commonly return HTTP **429 Too Many Requests**, optionally with **Retry-After**.

Main algorithms to remember: Fixed Window, Sliding Window Log, Sliding Window Counter, Token Bucket, Leaky Bucket.

Most interview-important: Token Bucket + distributed implementation + Redis atomicity + failure behavior + backoff/jitter.

2. What Is Rate Limiting?

Q: What is rate limiting?

Rate limiting is a mechanism that restricts how many requests a client can make to a service over a defined time period. For example, an API may allow 100 requests per minute per user. Requests beyond that limit are rejected, delayed, or otherwise controlled.

Q: Why do we need rate limiting?

To protect availability and expensive resources, prevent abuse, enforce fair usage, control API consumption, reduce accidental overload, and provide predictable service behavior during traffic spikes.

• Protect application servers and databases.

• Reduce brute-force and abuse attempts.

• Prevent one client from consuming disproportionate capacity.

• Control expensive operations such as search, report generation, login, or payment APIs.

• Support API plans such as free, paid, and enterprise tiers.

Q: What can we rate-limit?

Common dimensions are IP address, user ID, API key, session, device, tenant/organization, endpoint, region, or combinations such as user + endpoint.

```
Free user      -> 100 req/min
Premium user   -> 1,000 req/min
Admin          -> 10,000 req/min

POST /login        -> 5 req/min
GET /products      -> 1,000 req/min
POST /orders       -> 100 req/min
```

3. The Five Main Algorithms

Algorithm	Idea	Burst behavior	Memory	Key issue
Fixed Window	Counter per fixed time window	Can burst at boundary	Very low	Boundary burst
Sliding Window Log	Store request timestamps	Strict/accurate	High	Timestamp storage
Sliding Window Counter	Combine adjacent window counters	Approximate	Low	Approximation
Token Bucket	Tokens refill over time	Controlled bursts	Low	Atomic state updates
Leaky Bucket	Queue drains at controlled rate	Smooths traffic	Medium	Queue/backlog behavior

4. Fixed Window Counter

Divide time into fixed intervals, such as one-minute windows, and count requests inside each window.

```
Limit = 5 requests/minute

12:00:00 ---------------- 12:00:59
counter = 0

request -> counter = 1
request -> counter = 2
...
request -> counter = 5
request -> reject
```

Q: What is the advantage?

It is extremely simple, fast, and memory efficient. A single counter plus an expiration time can be enough.

Q: What is the major problem?

The boundary burst problem. A client can send the maximum number of requests at the end of one window and the maximum again immediately after the next window begins.

```
100 req/min limit

12:00:59 -> 100 requests
12:01:00 -> 100 requests

Potentially 200 requests in about 1 second.
```

Q: When is Fixed Window acceptable?

When simplicity and low cost matter more than strict smoothing, or when the API can tolerate boundary bursts.

5. Sliding Window Log

Store the timestamps of recent requests and examine only timestamps inside the current rolling window.

```
Limit = 5/minute

[10:00:01, 10:00:10, 10:00:20, 10:00:35, 10:00:50]

New request:
1. Remove timestamps older than 60 seconds.
2. Count remaining timestamps.
3. Allow if count < 5.
```

Q: Advantage?

High accuracy and no fixed-window boundary problem.

Q: Disadvantage?

It stores many timestamps. At large scale, memory usage and cleanup work can become expensive.

6. Sliding Window Counter

Approximate the rolling-window count using counters from the current and previous fixed windows.

```
Previous window = 80 requests
Current window  = 30 requests
We are 25% into current window.

Approximate rolling count
= 80 * 75% + 30
= 90
```

Q: Why use it?

It provides a useful compromise: much less memory than a timestamp log while reducing the boundary problem of a simple fixed window.

Q: What is the trade-off?

It is an approximation rather than an exact request-by-request view.

7. Token Bucket - Most Important

Imagine a bucket containing tokens. Each request consumes one or more tokens. Tokens are added at a fixed refill rate until the bucket reaches its maximum capacity.

```
Capacity = 10 tokens
Refill   = 2 tokens/sec

Initial: 10

5 requests:
10 -> 9 -> 8 -> 7 -> 6 -> 5

Wait 2 seconds:
5 + (2 * 2) = 9 tokens

If a request arrives:
9 -> 8
```

Q: Why is Token Bucket popular?

It separates burst capacity from sustainable rate. A client can use accumulated tokens for a short burst, while the refill rate controls the long-term average.

Example: capacity = 10 and refill rate = 2 tokens/sec means the client can potentially make a burst of 10 requests if the bucket is full, then continue at roughly 2 requests/sec as tokens refill.

Q: How are tokens calculated?

A typical implementation calculates elapsed time since the last refill, adds elapsed_time * refill_rate, caps tokens at bucket capacity, and then consumes the request cost.

```
tokens = min(
    capacity,
    tokens + (now - lastRefill) * refillRate
)

if tokens >= requestCost:
    tokens -= requestCost
    allow()
else:
    reject()
```

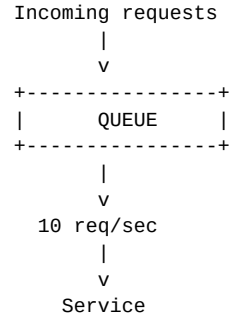
Q: What data do you store?

Usually the current token count and the last-refill timestamp. The key identifies the limited entity, such as user ID + endpoint.

```
rate:user123:/orders
{
  tokens: 73,
  lastRefill: 1726820000
}
```

8. Leaky Bucket

Requests enter a queue and leave at a controlled rate. If the queue is full, new requests may be rejected.



Q: How is it different from Token Bucket?

Token Bucket focuses on permission to send and can allow controlled bursts when tokens have accumulated. Leaky Bucket is commonly modeled as a queue that drains at a controlled rate, smoothing output.

Q: Where is Leaky Bucket useful?

When smoothing or shaping traffic is important, especially when a downstream system benefits from a steadier processing rate.

9. Token Bucket vs Leaky Bucket vs Windows

Property	Fixed Window	Sliding Log	Sliding Counter	Token Bucket	Leaky Bucket
Burst support	Poor at boundary	Low	Moderate	Controlled	Low
Accuracy	Medium	High	Approx.	High	High for output shaping
Memory	Very low	High	Low	Low	Medium
Typical use	Simple quotas	Strict rolling limit	Efficient rolling estimate	APIs / bursts	Traffic shaping
Main concern	Boundary burst	Storage	Approximation	Atomic state	Queue/backlog

10. Local vs Distributed Rate Limiting

Q: Why is a local in-memory limiter incorrect in a multi-server system?

Because each application server has its own counter. A user can distribute requests across servers and exceed a limit that was intended to be global.

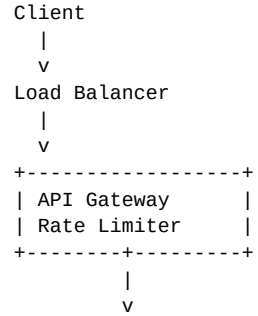
```
Limit = 100 req/min

Server 1 -> 100
Server 2 -> 100
Server 3 -> 100

User could potentially get ~300 requests/min.
```

Q: How do you solve it?

Use shared state or a centralized/distributed rate-limiting mechanism. Redis is a common choice because it is fast, supports atomic primitives, and supports expiration.



```
   +-----------+
   |   Redis   |
   +-----------+
         |
         v
 Application
```

11. Redis and Race Conditions

Q: Why can a naive GET -> check -> SET implementation fail?

Two concurrent requests can read the same old value before either writes the new value. Both may conclude that capacity is available and both may be accepted.

```
Request A: GET -> 0
Request B: GET -> 0

A: 0 < 1 -> allow
B: 0 < 1 -> allow

But limit was 1.
```

Q: How do you prevent this?

Use atomic Redis operations or an atomic Lua script when multiple reads/calculations/writes must happen as one operation.

For a simple counter, commands such as INCR are atomic. For Token Bucket, the refill calculation, capacity check, token deduction, and state update generally need to be performed atomically.

Q: Why Lua?

A Redis Lua script executes atomically on the Redis server, allowing several dependent operations to behave as one indivisible operation.

12. HTTP Behavior

Q: What status code should a rate-limited request receive?

429 Too Many Requests is the standard response for exceeding a request rate.

```
HTTP/1.1 429 Too Many Requests
Retry-After: 30
Content-Type: application/json

{
  "error": "rate_limit_exceeded"
}
```

Q: What is Retry-After?

It tells the client when it should retry. It may be expressed as a delay or a date, depending on the HTTP response format.

Q: Should clients retry immediately?

Usually no. A well-behaved client should use backoff and, for distributed clients, jitter to avoid synchronized retry spikes.

13. Exponential Backoff + Jitter

```
Attempt 1 -> wait ~1 sec
Attempt 2 -> wait ~2 sec
Attempt 3 -> wait ~4 sec
Attempt 4 -> wait ~8 sec
Attempt 5 -> wait ~16 sec
```

Add random jitter so clients do not all retry at exactly the same moment.

Q: What problem does jitter solve?

The thundering-herd problem: if a very large number of clients receive the same retry delay and retry simultaneously, the retry itself can create another traffic spike.

14. What Should Be the Rate-Limit Key?

The key should match the abuse/fairness boundary you care about.

Key	Useful for	Caution
IP	Anonymous/public APIs	NAT can group many legitimate users; attackers can rotate IPs
User ID	Authenticated APIs	Requires authentication
API key	Developer APIs	Keys can be shared/leaked
Tenant/org	Multi-tenant SaaS	Large tenants may need separate quotas
Endpoint	Protect expensive operations	More state/logic
User + endpoint	Fine-grained protection	More keys and complexity
IP + account	Login/security controls	Needs careful tuning to avoid false positives

15. Multi-Level Rate Limiting

Real systems often apply several limits simultaneously.

```
Global system limit
       |
       v
Tenant / organization limit
       |
       v
User limit
       |
       v
Endpoint limit
       |
       v
IP / abuse protection
```

Example: global 100K req/sec, tenant 10K req/sec, user 100 req/sec, and login endpoint 5 req/min. A request may need to pass every applicable limit.

16. Rate Limit vs Quota vs Concurrency Limit vs Throttling

Concept	Question it answers	Example
Rate limit	How many requests over time?	100 requests/minute
Quota	How much total usage in a longer period?	1 million requests/month
Concurrency limit	How many are executing simultaneously?	20 concurrent requests
Throttling	How do we slow/control traffic under pressure?	Process at 50 req/sec

These concepts can be combined. For example: 100 req/min + 1M/month + 20 concurrent operations.

17. Where Should the Rate Limiter Live?

Often as close to the edge as practical, such as a CDN, API gateway, or load balancer layer, because rejected requests should not consume application/database resources.

```
Client
  |
  v
```

```
CDN / WAF (optional)
  |
  v
API Gateway / Rate Limiter  <-- common place
  |
  v
Application
  |
  v
Database
```

Q: Should we always rate-limit only at the gateway?

Not necessarily. Defense-in-depth can use gateway limits plus service-level limits for especially expensive or sensitive operations.

18. Endpoint-Specific Limits

```
GET  /products       -> 1,000/min
POST /orders         ->   100/min
POST /login          ->     5/min
POST /forgot-password->     3/min
POST /reports        ->    20/min
```

Expensive and security-sensitive endpoints often need stricter limits than cheap read-only endpoints.

19. Rate Limiting for Login and Abuse Protection

Q: Is IP-only limiting enough for login protection?

No. Attackers can rotate IP addresses, while legitimate users may share an IP through NAT or corporate networks. A layered approach can consider account identity, IP, device/session signals, endpoint, and other abuse controls.

- Credential stuffing
- Brute-force attempts
- Password spraying
- Automated scraping

Security limits should be designed carefully so they reduce abuse without unnecessarily locking out legitimate users.

20. What If Redis Goes Down?

Q: Fail open or fail closed?

Fail open: allow requests if the limiter cannot be consulted. This favors availability but can temporarily remove protection. **Fail closed:** reject when the limiter is unavailable. This favors strict protection but can cause an outage. A hybrid policy is often more practical.

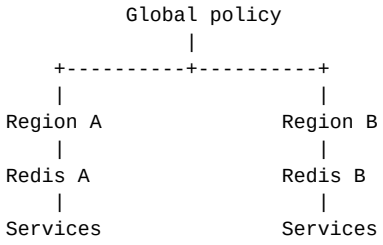
Policy	Benefit	Risk
Fail open	Higher availability	Potential abuse / overload
Fail closed	Strong protection	Rate limiter outage can block traffic
Hybrid	Different behavior by endpoint/risk	More policy complexity

For example, a low-risk read endpoint might tolerate a fallback, while a sensitive or very expensive operation may use a stricter fallback. The correct policy depends on business and reliability requirements.

21. Scaling a Distributed Rate Limiter

Q: What if Redis becomes the bottleneck?

Possible strategies include Redis Cluster/sharding, careful key distribution, regional limiters, local fast-path checks where safe, hierarchical limits, and a dedicated rate-limiting service.



Q: What is the trade-off with multi-region rate limiting?

A truly global limit requires coordination across regions, which can add latency and consistency complexity. Regional limits are simpler and faster but do not automatically enforce one exact global counter.

22. Millions of Users and Key Management

Use expiration/TTL for temporary rate-limit state so inactive users do not leave permanent keys.

```
rate:user:123
TTL = 60 seconds
```

Also consider key cardinality, memory usage, hot keys, endpoint combinations, and whether every request really needs a unique state entry.

23. Weighted / Cost-Based Rate Limiting

Not every request needs to cost one token. Expensive operations can consume more tokens.

```
GET /health        -> cost 1
GET /products      -> cost 1
POST /report       -> cost 10
POST /export       -> cost 20
```

This is useful when request cost differs substantially and you want the limiter to protect a scarce resource rather than simply count HTTP requests.

24. What Should Happen After the Limit?

Action	When useful	Trade-off
Reject with 429	Immediate protection	Client must retry
Queue	Work can wait	Adds latency and queue capacity
Backoff/retry	Transient overload	Can amplify traffic without jitter
Degrade	Non-critical features	More application complexity

25. Design a Distributed Rate Limiter - Interview Framework

Step 1: Clarify the requirement

Ask: What is being limited - user, IP, API key, tenant, endpoint? What is the limit? Is it global or regional? Are bursts allowed? What happens when exceeded? What availability/latency requirement exists?

Step 2: Choose the algorithm

Use Fixed Window for simple low-cost limits, Sliding Log for exact rolling behavior, Sliding Counter for an efficient approximation, Token Bucket when controlled bursts matter, and Leaky Bucket when smoothing output is important.

Step 3: Choose the placement

Put it at the gateway/edge when possible so rejected requests do not consume application resources.

Step 4: Choose state

For multiple servers, use shared state such as Redis or a dedicated rate-limiting service. Decide how state is partitioned and expired.

Step 5: Make updates atomic

Avoid GET-check-SET races. Use atomic counters or server-side atomic scripts for multi-step algorithms.

Step 6: Define failure behavior

Explicitly choose fail-open, fail-closed, or hybrid behavior.

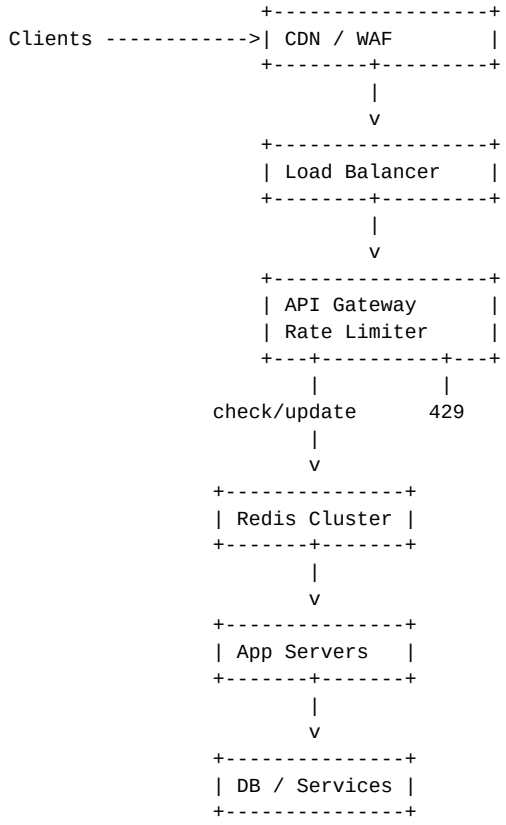
Step 7: Define client behavior

Return 429 and, when useful, Retry-After. Use exponential backoff and jitter for retries.

Step 8: Discuss scaling and observability

Cover sharding, hot keys, multi-region behavior, metrics, logs, alerts, and capacity planning.

26. Reference Architecture



27. Monitoring and Observability

A production rate limiter should be observable. Useful metrics include:

• Requests allowed.

• Requests rejected with 429.

• Rate-limit decisions by endpoint, tenant, user class, and region.

• Redis latency/error rate.

• Limiter decision latency.

• Token/bucket utilization where meaningful.

• Queue depth for queue-based designs.

• Hot keys / unusually concentrated traffic.

• Fallback/fail-open/fail-closed events.

Logs should avoid exposing sensitive identifiers unnecessarily. Prefer stable internal IDs or appropriately protected identifiers.

28. Interview Questions - Ready-to-Say Answers

Q: Explain rate limiting in simple words.

Rate limiting puts a controlled limit on how frequently a client can use a service. For example, 100 requests per minute per user. It protects the system from overload and abuse and helps enforce fair usage.

Q: Why not just let the server handle every request?

Because traffic can be bursty or abusive. Without a limit, one client or a sudden spike can consume CPU, memory, database connections, network bandwidth, or downstream quotas and affect other users.

Q: What are the common rate-limiting algorithms?

Fixed Window Counter, Sliding Window Log, Sliding Window Counter, Token Bucket, and Leaky Bucket.

Q: Which algorithm would you choose for an API that should allow bursts?

Token Bucket is a natural choice because it allows accumulated tokens to be consumed as a short burst while the refill rate controls sustained usage.

Q: Which algorithm has the boundary problem?

Fixed Window Counter. A client can use the full quota near the end of one window and again immediately after the next window starts.

Q: Which algorithm stores timestamps?

Sliding Window Log.

Q: Why not always use Sliding Window Log?

Because storing every request timestamp can become memory-heavy at large scale.

Q: What is Sliding Window Counter?

It estimates the rolling count from counters in adjacent windows. It uses much less memory than a timestamp log but is approximate.

Q: Explain Token Bucket.

A bucket contains tokens up to a maximum capacity. Tokens refill at a fixed rate. A request consumes tokens. If enough tokens are available, the request is allowed; otherwise it is rejected or delayed.

Q: What controls burst size in Token Bucket?

Bucket capacity. A full bucket represents the maximum accumulated burst that can be consumed immediately, subject to request cost.

Q: What controls sustained rate?

The refill rate.

Q: Explain Leaky Bucket.

Requests enter a queue and are released at a controlled rate. This smooths traffic and can protect downstream systems from bursts.

Q: Token Bucket vs Leaky Bucket?

Token Bucket controls permission and can allow controlled bursts. Leaky Bucket is commonly modeled as a queue that drains at a controlled rate, smoothing output.

Q: Why use Redis for distributed rate limiting?

It is fast, supports atomic operations and expiration, and can be shared by multiple application servers.

Q: Why is GET-check-SET dangerous?

Concurrent requests can read the same value and both decide that capacity exists, causing the system to exceed the limit.

Q: How do you make a Token Bucket atomic?

Perform refill calculation, capacity check, deduction, and state update as one atomic operation, commonly using a Redis Lua script or another atomic server-side mechanism.

Q: What status code is returned when the limit is exceeded?

HTTP 429 Too Many Requests.

Q: What is Retry-After?

A response header that tells the client when it should retry.

Q: Why use exponential backoff?

To reduce repeated pressure on an overloaded service by increasing the wait time between retries.

Q: Why add jitter?

To randomize retry timing and prevent many clients from retrying simultaneously and creating a synchronized traffic spike.

Q: How do you rate-limit multiple application servers?

Use shared distributed state or a centralized limiter. Otherwise each server may maintain a separate quota and a client can exceed the intended global limit by spreading requests across servers.

Q: Where should the rate limiter be placed?

Often at the API gateway or edge so rejected requests do not consume application resources. Additional service-level limits can protect particularly expensive operations.

Q: IP-based or user-based limiting?

It depends on the requirement. IP is useful for anonymous traffic and abuse control, while user/API-key limits are better for authenticated usage. Layered keys can address different abuse patterns.

Q: What happens if Redis is down?

Choose a deliberate policy: fail open for availability, fail closed for strict protection, or a hybrid policy based on endpoint risk. The important point is to make the behavior explicit.

Q: How would you rate-limit login?

Use layered controls rather than relying only on IP. Consider account identity, IP, device/session signals, endpoint, and appropriate security controls, while avoiding excessive lockouts of legitimate users.

Q: How do you handle different API costs?

Use weighted tokens or request costs. A cheap request may cost one token while an expensive export or report may cost ten or twenty.

Q: What is quota vs rate limit?

A rate limit controls usage over a short time interval, such as 100 requests/minute. A quota controls total allowed usage over a longer period, such as 1 million requests/month.

Q: What is concurrency limiting?

It limits the number of requests that may execute at the same time, such as at most 20 concurrent expensive operations. It is different from a rate limit, which controls requests over time.

Q: What is throttling?

Throttling is the broader idea of controlling or slowing traffic, especially under load. Rate limiting is one common mechanism used to do that.

Q: How would you scale a rate limiter to millions of users?

Use distributed/shared state, partition keys across Redis shards, apply TTLs to temporary state, avoid unnecessary key cardinality, monitor hot keys, and consider regional or hierarchical designs.

Q: What is a hot key?

A key receiving disproportionately many operations, for example a single heavily shared API key or tenant. It can overload one shard even if overall cluster traffic is acceptable.

Q: Global vs regional rate limiting?

Global limits require cross-region coordination and can add latency and consistency complexity. Regional limits are simpler and faster but enforce regional rather than one exact global counter unless additional coordination exists.

Q: How would you test a rate limiter?

Test under the limit, exactly at the limit, just above the limit, boundary times, concurrent requests, bursts, clock differences, retries, Redis failures, shard failures, and high-cardinality traffic.

29. Coding / Implementation Questions

Q: Write a simple Fixed Window algorithm.

```
function allow(key, limit, windowSeconds) {
    const count = redis.incr(key);

    if (count === 1) {
        redis.expire(key, windowSeconds);
    }

    return count <= limit;
}
```

Interview note: explain that production implementations must consider atomicity and TTL edge cases. For more complex behavior, use an atomic server-side script.

Q: Write Token Bucket pseudocode.

```
function allow(state, now):
    elapsed = now - state.lastRefill
    state.tokens = min(
        capacity,
        state.tokens + elapsed * refillRate
    )
    state.lastRefill = now

    if state.tokens >= cost:
        state.tokens -= cost
        return ALLOW

    return REJECT
```

For distributed execution, this state transition should be atomic.

30. Testing Checklist

Test	Expected concern
0 requests	Allow
Exactly at limit	Define whether the boundary request is allowed
One above limit	Reject/delay
Burst at window boundary	Check fixed-window boundary behavior
Concurrent requests	No double-allow beyond limit
Multiple servers	Global limit remains correct
Redis latency	Decision latency remains acceptable
Redis unavailable	Configured fallback policy is followed
Clock skew	Algorithm does not produce unsafe behavior
Expired keys	State is cleaned up
Hot tenant/user	No single shard becomes overloaded
Retry storm	Backoff + jitter prevent synchronized retries

31. Final 2-Minute Cheat Sheet

Definition: Rate limiting controls request frequency to protect systems and enforce fair usage.

429: Too Many Requests.

Fixed Window: simple counter; boundary burst problem.

Sliding Log: exact rolling view; stores timestamps; memory-heavy.

Sliding Counter: efficient approximate rolling count.

Token Bucket: tokens refill; supports controlled bursts; capacity = burst size; refill rate = sustained rate.

Leaky Bucket: queue drains at controlled rate; smooths traffic.

Distributed: shared state, commonly Redis.

Atomicity: avoid GET-check-SET races; use atomic commands/Lua scripts.

Placement: gateway/edge when possible.

Failure: decide fail-open vs fail-closed vs hybrid.

Retry: exponential backoff + jitter.

Scale: shard, TTL, avoid hot keys, consider regional/hierarchical limits.

Security: don't rely only on IP; combine appropriate dimensions.

Advanced: weighted tokens, concurrency limits, quotas, multi-level limits, multi-region consistency.

32. A Strong System-Design Closing Statement

A strong way to close a rate-limiter design answer is:

> ***"I would first clarify the limiting dimension, request rate, burst requirement, scope, and failure policy. For a distributed API, I would put the limiter near the gateway, use a shared store such as Redis, and make the state transition atomic. If bursts are required, I would consider Token Bucket. I would return 429 with Retry-After when appropriate, use exponential backoff with jitter on clients, and then discuss sharding, hot keys, TTLs, multi-region behavior, observability, and what happens if the shared store fails."***

End of guide - use this as your single-place rate-limiter revision sheet before backend and system-design interviews.